

1- Write an expression for the clock period and the hold-time condition when there is a negative skew observed between two registers.

1) clock period

$$T - t_{\text{skew}} \geq t_{\text{cq}} + t_{\text{pd}} + t_{\text{sw}}$$

2) Hold-time

$$t_{\text{hold}} - t_{\text{skew}} \leq t_{\text{cq}} + t_{\text{pd}}$$

2- Explain briefly why you would want to have negative skew instead of positive skew if you had the option.

The skew is relatively large when compared to the Hold-Time requirement. Thus, the skew impact on the Hold-Time Requirement is more sensible and effective compared to its impact on the Setup time, and as the -ve skew relaxes the Hold time Requirement, it would be better to have negative skew if we have the option but we have to make sure that it won't result in Setup time violations.

3- Explain why jitter has no impact on hold time condition

The jitter affects the same clk edge at all of the Registers Simultaneously. Thus, it has no impact on the Hold time Condition which is checked at the same clk edge.

4-Write an expression for the clock period of a pipeline suffering from jitter

Considering the worst case Scenario at which the 1st edge jitter is +ve & the 2nd one is -ve, the clk period is given by:

$$T - 2\pm_j > t_{cq} + t_{pd} + t_{su}$$

5-Write one **disadvantage** of the grid method for clock distribution

- 1) it minimizes the absolute delay but not the Relative one.
- 2) Large Area & Power Consumption

6-Write one **disadvantage** for the tree method for clock distribution

Not design independent

7- This piece of code includes one mistake. This mistake will not produce an error (neither in the compiler nor the synthesizer). Describe why this mistake is problematic and write down the code with the mistake solved. Draw the logic circuit (high level circuit like decoder, encoder, etc) represented by the fixed code.

Original code	Fixed code
<pre> Process(sel) Begin If (sel="00") then A<='0'; B<='1'; C<='0'; Elsif (sel="01") then A<='0'; D<='0'; C<='0'; Elsif (sel="10") then A<='1'; B<='1'; C<='1'; Else A<='1'; B<='0'; C<='1'; End if; End process; </pre>	

Why original code is bad (what it would produce):

this process is a Combinational one. However, there are missing assignments for the signals 'B' and 'D'. Thus, the synthesis tool infers a latch for 'B' and 'D'.

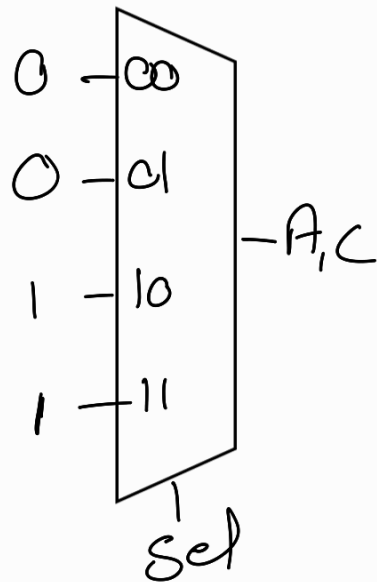
the fixed code is :-

```

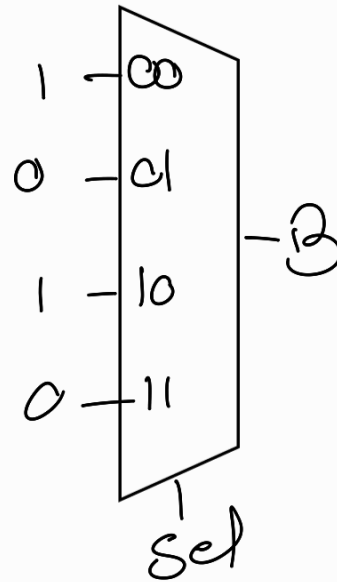
Process(sel)
Begin
  If (sel="00") then
    A<='0'; B<='1'; C<='0'; D<='0';
  Elsif (sel="01") then
    A<='0'; D<='0'; C<='0'; B<='0';
  Elsif (sel="10") then
    A<='1'; B<='1'; C<='1'; D<='0';
  Else
    A<='1'; B<='0'; C<='1'; D<='0';
  End if;
End process;

```

- * Signal D is always Zero
- * a 4x1 MUX is used to generate A, B, C



$$A, C = \text{sel}(1)$$



$$B = \text{sel}(0)'$$

8- Show all your work for this question. Find the percentage improvement in the critical path delay of an adder when moving from a ripple carry adder to a carry bypass adder architecture. The adder is 40 bits. In the carry-bypass adder, there are 5 blocks each of 8-bits. Use the following:

$$T_{sum} = 0.69 \cdot 30 R_{oCo}$$

$$T_{carry} = 0.69 \cdot 18 R_{oCo}$$

$$T_{mux} = 0.69 \cdot 4 R_{oCo}$$

Delay of 40-bit ripple carry adder:

$$\begin{aligned} t_{RCA} &= 39 t_c + t_{sum} \\ &= 0.69 (732 R_{oCo}) \end{aligned}$$

Delay of 40-bit carry-bypass adder:

$$\begin{aligned} t_{Bypass} &= 8 t_c + 4 t_{mux} + 7 t_c + t_s \\ &= 0.69 (316 R_{oCo}) \end{aligned}$$

Percentage improvement:

$$= \frac{732 - 316}{732} \times 100 = 56.831\%$$

9- Justify why we can pass the read and write pointers in an asynchronous FIFO through synchronizers even though they are busses rather than signals.

Passing Busses through Synchronizers corrupt the data due to the one cycle uncertainty in the Double Flip Flop Synchronizers. However, in the case of the Read & write pointers in the Async FIFO, they are Gray encoded so the transition is guaranteed to be in a single bit position. Thus, they can be passed through synchronizers.

10- What is the price you pay for keeping wires thick when scaling technology (wire thickness scaling down slower than width and length).

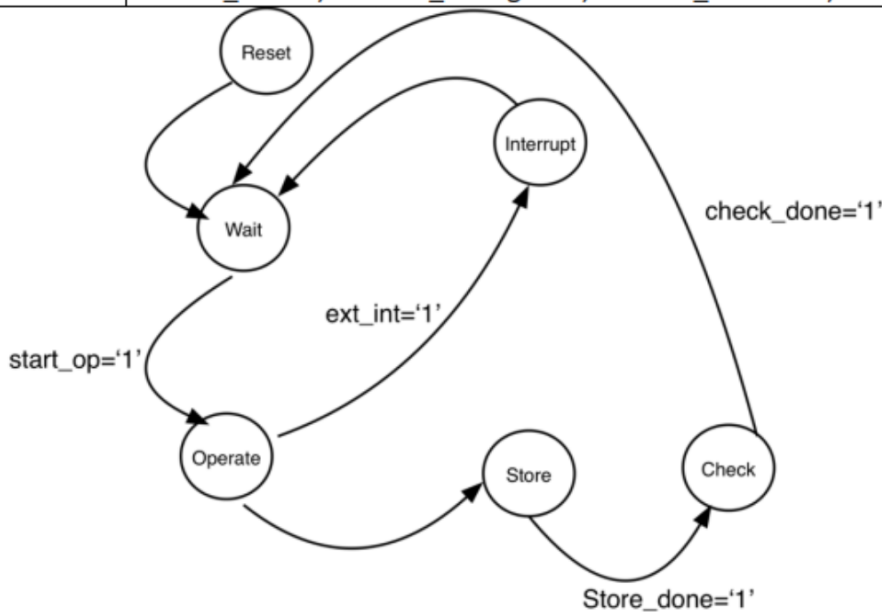
* the inter-wire capacitance will scale down slower than its width and length

as the inter-wire capacitance is given by:-

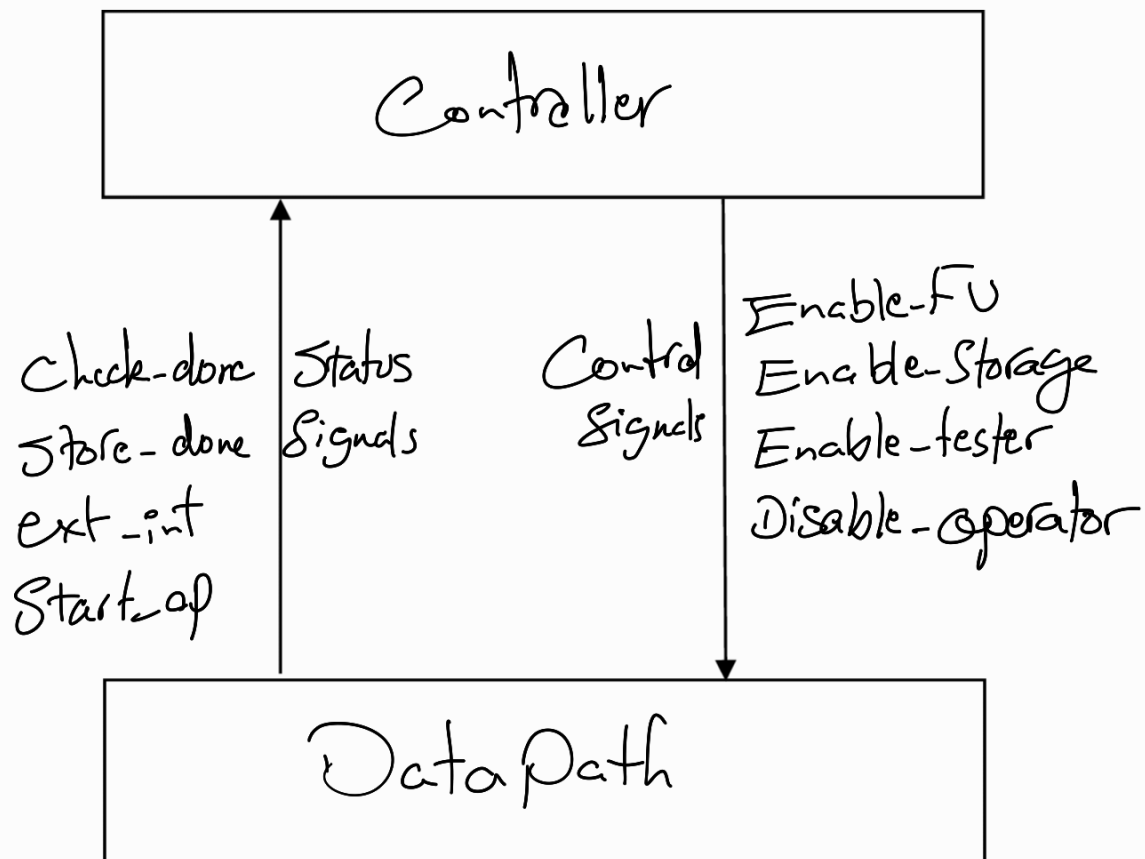
$$C_{\text{inter-wire}} = \frac{\epsilon \times t_w L}{L_{\text{sep}}}$$

11- The state diagram below describes a very simple state machine. The table shows the outputs of the state machine for each state.

State	Signals
Reset	Enable_FU='0'; Enable_storage='0'; Enable_tester='0'; Disable_operator='1'
Wait	Enable_FU='0'; Enable_storage='0'; Enable_tester='0'; Disable_operator='0'
Interrupt	Enable_FU='0'; Enable_storage='0'; Enable_tester='0'; Disable_operator='1'
Operate	Enable_FU='1'; Enable_storage='0'; Enable_tester='0'; Disable_operator='0'
Store	Enable_FU='0'; Enable_storage='1'; Enable_tester='0'; Disable_operator='0'
Check	Enable_FU='0'; Enable_storage='1'; Enable_tester='1'; Disable_operator='1'



Draw a block diagram showing the status signals going from the datapath to the state machine and the control signals going from the state machine to the controller.



12- Explain why there are no arrows going into the reset state in question 11, but we still can sometimes go to the reset state.

Because the reset state condition is well defined & known
which is the async. Reset assertion

13- Still with question 11, write the code of the next state process of the state machine (the combinational process that determines the next state based on the current state and status signals)

```
next_state_process : process( current_state, start_op, ext_int, store_done, check_done )
begin
  case( current_state ) is
    when reset =>
      next_state <= wait;
    when wait =>
      if start_op = '1' then
        next_state <= operate;
      else
        next_state <= wait;
      end if ;
    when operate =>
      if ext_int = '1' then
        next_state <= interrupt;
      else
        next_state <= store;
      end if ;
    when store =>
      if store_done = '1' then
        next_state <= check;
      else
        next_state <= store;
      end if ;
    when check =>
      if check_done = '1' then
        next_state <= wait;
      else
        next_state <= check_done;
      end if ;
    when interrupt =>
      next_state <= wait;
    when others =>
      next_state <= reset;
  end case ;
end process ; -- next_state_process
```

14- When moving between two technologies, dimensions scale down by a factor K ($L, W, \dots \rightarrow L/K, W/K, \dots$), voltages by a factor U ($V_{dd}, V_{th}, \dots \rightarrow V_{dd}/U, V_{th}/U, \dots$), and the die dimension grows by a factor K_w . Find an expression for the scaling of the resistance of a global wire

$$R = \frac{L}{\sigma A} = \frac{L}{\sigma w t_w} \propto K_w K^2$$

15-Find the scaling factor of power dissipation in a global wire ($P=IV, V=IR$). Use the givens in question 14.

$$P_{\text{global wire}} = \frac{V^2}{R} \Rightarrow P_{\text{new global wire}} = \frac{1}{U^2} \times \frac{1}{K^2 K_w} P_{\text{global wire}}$$

16-In a synchronizer, when can the receiver lower the value of the ack signal. Why can't it lower it at will?

The Receiver can lower the value of the ack signal after detecting a falling edge on the Synchronized Req. signal sent from the Transmitter.

If the Receiver lowers the ack signal at will, the ack signal pulse isn't guaranteed to be Synchronized at the Transmitter side. So the Tx won't detect that the Receiver has read the data Successfully. Thus, the Tx won't update the data Bus with new data.

17-Write the expression of critical path delay for the carry-bypass adder in terms of T_{mux} , T_{carry} , T_{sum} , K (number of bits per block), and P (number of blocks). Find the optimal number of bit bits per block (hint: differentiate)

Expression:

$$T_{\text{carry Bypass}} = Kt_c + (P-1)t_{mux} + (K-1)t_c + t_{sum}$$
$$= Kt_c + \left(\frac{N}{K} - 1\right)t_{mux} + (K-1)t_c + t_{sum}$$

Optimum :-

diff. $T_{\text{carry Bypass}}$ w.r.t K

$$\frac{\partial T}{\partial K} = t_c - \frac{N}{K^2}t_{mux} + t_c = 0$$

$$\therefore \sqrt{K_{\text{Optimum}}} = \sqrt{\frac{N}{2} \times \frac{t_{mux}}{t_c}}$$

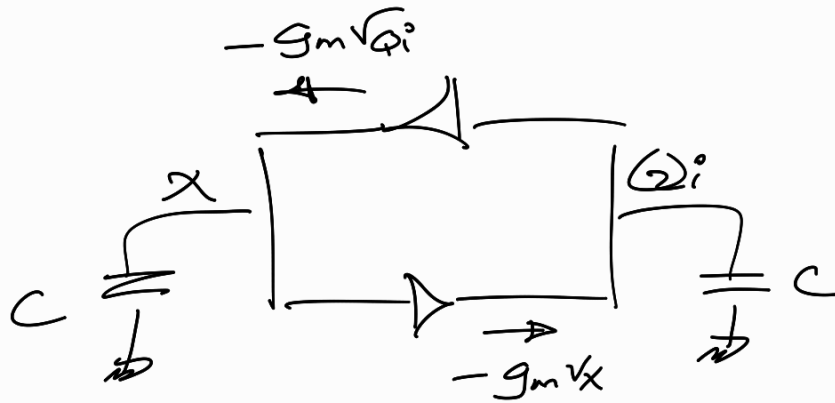
18-If multiplexer delay increases in a carry-bypass adder, would this increase the number of bits per block, or increase the number of blocks? Explain. (You may want to refer to question 17).

if t_{mux} increases, the optimum bits per block increases. as the mux delay inc, the overhead due to passing Different blocks inc w.r.t (t_c).
Thus, we have to decrease the total number of blocks by increasing K .

19- Show that the time to exit metastability is:

$$T_{exit} = \tau \ln(V_{DD}/V_d(0))$$

Start from the sketch of the two inverters as amplifiers



$$(1) -g_m v_i = C \frac{dv_x}{dt}$$

$$(2) -g_m v_x = C \frac{dv_i}{dt}$$

$$\text{By sub. (1)(2)} \rightarrow -g_m v_d = C \frac{dv_d}{dt}$$

$$\Rightarrow v_d = v_d(0) e^{+t/\tau}, \quad \tau = C/g_m$$

$$\Rightarrow t = \tau \ln\left(\frac{v_d}{v_d(0)}\right)$$

$$\Rightarrow T_{exit} = t|_{v_d=v_{dd}} = \tau \ln\left(\frac{V_{dd}}{v_d(0)}\right)$$

* Entity, Component & Instance

Entity: it is the Basic Building Block of VHDL.

The entity is a description of the inputs and outputs of a component without any concern to what lies within it. $\rightarrow$ Black Box

Component: the component declaration is used to declare kinds of entities to be used to form a larger circuit using the keyword "Component".

The components are declared in the same space where internal signals are declared.

Instance: the instance is used to create a copy of any component to be used in the body of the architecture. if multiple instances are declared for the same component, they are distinguished using labels.

Each instance is followed by a "port map" indicating the connections of the ports of the component in this particular instance.

* Point out the Difference between the Three

- 1) the entity describes the inputs and outputs of any component to be used to build a larger circuit, it is also used as a top level wrapper.
- 2) the components define the kinds of blocks which will be used in the architecture (Adder, Multiplier, Encoder...) without specifying how many adders are used or their any connections
- 3) instances are used to declare copies of each component, we may use 3 adders in our design for instance. Instances define the connections of each component copy in addition to the generic map if req.